TALLER PRÁCTICO GRUPAL

Introducción a Git: Instalación, Configuración y Repositorio Local

Docente: Jhonathan Arley Peñaloza Sierra

Integrantes del grupo:	Valeria Lizcano Zlatan Villamizar Julio Castaño
Fecha:	

Objetivo del taller

Practicar la instalación, configuración y uso básico de Git como sistema de control de versiones. Al finalizar, cada grupo habrá creado su primer repositorio local, realizado commits y explorado el historial de cambios.

Instrucciones generales

Formen grupos de 3 a 4 estudiantes. Cada grupo debe trabajar en un mismo computador. Sigan los ejercicios en orden, ya que cada parte construye sobre la anterior. Todos los integrantes deben participar. Entreguen este documento completo al finalizar.

Requisitos previos

Tener un computador con acceso a internet para descargar Git. Se recomienda tener Visual Studio Code instalado. No se necesita experiencia previa con Git.

Parte 1: Entendiendo los Conceptos de Git

Ejercicio 1.1 – ¿Qué es Git?

Respondan con sus propias palabras las siguientes preguntas basándose en lo visto en clase:

a) ¿Qué es un Sistema de Control de Versiones Distribuido (SCVD)? ¿Para qué sirve?

Un **Sistema de Control de Versiones Distribuido (SCVD)** es una herramienta donde cada persona que trabaja en un proyecto tiene una copia completa del historial y de los archivos en su propia computadora, no solo en un servidor central.

Colaboración total: Permite que varios desarrolladores trabajen en el mismo código al mismo tiempo sin estorbarse.

- **Seguridad y respaldo:** Si el servidor principal falla, cualquier copia local de un miembro del equipo puede usarse para restaurar el proyecto completo.

- **Trabajo sin conexión:** Puedes hacer cambios, registrar avances y revisar el historial sin necesidad de estar conectado a internet.

---

b) Mencionen al menos 3 cosas que permite hacer un sistema de control de versiones:

Regresar en el tiempo: Puedes revertir el proyecto a una versión anterior si cometes un error grave o si algo deja de funcionar.

- **Crear Ramas (Branches):** Permite crear "líneas temporales" separadas para experimentar con funciones nuevas sin alterar el código que ya funciona.
- **Registro de cambios (Auditoría):** Mantiene un historial detallado que muestra exactamente qué se cambió, cuándo se hizo y quién fue el autor de cada modificación.

Ejercicio 1.2 – Relacionar conceptos

Unan cada concepto de Git con su definición correcta. Escriban la letra correspondiente en la columna derecha:

> **Definiciones disponibles**
>
> A) Copia de un repositorio remoto a tu máquina local | B) Instantánea del proyecto en un momento determinado | C) Versión del repositorio que diverge del desarrollo principal | D) Área intermedia donde se colocan cambios antes de confirmarlos | E) Enviar cambios locales a un repositorio remoto | F) Propuesta de cambios para que el propietario revise y fusione | G) Archivo que especifica qué archivos Git debe ignorar | H) Incorporar cambios de una rama a otra

Concepto	Letra de la definición
`Commit`	Es una captura o "foto" del estado de tus archivos en un momento específico. Al hacer un commit, guardas permanentemente los cambios en el historial con un mensaje explicativo.
`Branch`	Es una línea de tiempo paralela. Sirve para desarrollar funciones nuevas o corregir errores sin afectar el código principal (la rama `main`).
`Staging Area`	Es el paso intermedio antes del commit. Aquí seleccionas qué cambios específicos quieres incluir en tu próxima "foto" del proyecto usando el comando `git add`.
`Push`	Es el proceso de subir tus commits locales a un repositorio remoto (como GitHub o GitLab) para que otros puedan verlos y usarlos.

`Clone`	Es la acción de descargar una copia completa de un repositorio remoto a tu computadora local por primera vez, incluyendo todo su historial.
`Merge`	Es la unión de dos ramas. Se usa para integrar los cambios que hiciste en una rama secundaria dentro de la rama principal.
`Pull Request`	Es una petición para que alguien revise tus cambios antes de integrarlos al proyecto principal. Es común en entornos de trabajo colaborativo para asegurar la calidad del código.
`.gitignore`	Es un archivo de texto que le indica a Git qué archivos o carpetas debe ignorar y nunca rastrear (por ejemplo, archivos de configuración personal, contraseñas o carpetas de librerías pesadas).

Parte 2: Instalación y Configuración de Git

Ejercicio 2.1 – Instalar Git

Sigan los pasos de instalación según su sistema operativo (vistos en clase). Cuando terminen, verifiquen que Git está instalado.

a) Abran una terminal (CMD, PowerShell o Terminal de Linux) y escriban el siguiente comando:

```
git --version
```

b) ¿Qué versión de Git les aparece? Escríbanla aquí:

git version 2.34.1

c) ¿Tuvieron algún problema durante la instalación? Si es así, descríbanlo y cómo lo resolvieron:

no

Ejercicio 2.2 – Configuración inicial

Ahora van a configurar Git con su nombre de usuario y correo electrónico. Estos datos aparecerán en cada commit que hagan.

a) Ejecuten los siguientes comandos en la terminal, reemplazando los datos con los suyos:

```
# Configurar nombre de usuario
git config --global user.name "SuNombreDeUsuario"

# Configurar correo electrónico
git config --global user.email "sucorreo@ejemplo.com"

# (Opcional) Configurar VS Code como editor
git config --global core.editor "code --wait"
```

b) ¿Con qué nombre y correo configuraron Git? Escríbanlos:

arenasvaleria030-cmyk
arenasvaleria030@gmail.com

Recuerden

Se aconseja tener una cuenta de usuario en GitHub para facilitar la colaboración y la conexión remota con repositorios en la plataforma.

Parte 3: Creando su Primer Repositorio Local

Ejercicio 3.1 – Crear un proyecto e inicializar Git

Van a crear una carpeta de proyecto y a inicializar Git en ella. Sigan estos pasos exactamente:

Paso 1: Crear la carpeta del proyecto

Creen una carpeta en el escritorio llamada "mi-primer-repo". Pueden hacerlo desde el explorador de archivos o con la terminal.

Paso 2: Abrir la carpeta en VS Code

Abran Visual Studio Code y abran la carpeta que acaban de crear (Archivo → Abrir carpeta).

Paso 3: Abrir la terminal integrada

En VS Code, abran la terminal integrada (menú Terminal → Nueva terminal, o Ctrl + Ñ).

Paso 4: Inicializar Git

Escriban el siguiente comando:

```
git init
```

a) ¿Qué mensaje les apareció en la terminal después de ejecutar git init?

Initialized empty Git repository in [ruta/de/tu/carpeta/.git/]

b) ¿Qué cambios observan en VS Code después de inicializar Git? (Miren la barra inferior y la barra lateral)

Aparición de números
Letra "U"
Nombre de la rama
Icono de la rama
Líneas de color

> **Dato importante**
> Después de git init, aparece la palabra "main" en la barra inferior. Es la rama principal que Git crea por defecto. Los archivos aparecen con la letra U (Untracked) porque Git aún no los está rastreando.

Ejercicio 3.2 – Crear archivos para el proyecto

Ahora van a crear 3 archivos dentro de su proyecto para tener contenido con el cual practicar:

a) Creen los siguientes archivos en VS Code (Archivo → Nuevo archivo):

Archivo	Contenido sugerido
README.md	`# Mi Primer Repo\nEste es mi primer proyecto con Git.`
index.html	`<html><body><h1>Hola Git</h1></body></html>`
notas.txt	`Aquí van mis notas de clase sobre Git.`

b) Después de crear los archivos, ejecuten en la terminal:

```
git status
```

c) ¿Qué muestra git status? ¿De qué color aparecen los archivos? ¿Qué significa eso?

Te da un resumen del estado actual del repositorio

Rojo: Significa que los archivos están en el **Working Directory** (Directorio de trabajo) pero no han sido agregados al área de preparación.

Parte 4: Gestionando el Staging Area

Ejercicio 4.1 – Agregar archivos al Staging Area

Ahora van a mover archivos al área de preparación (staging area) para prepararlos antes de hacer un commit.

a) Agreguen SOLO el archivo README.md al staging area:

```
git add README.md
```

b) Ejecuten git status de nuevo. ¿Qué diferencia ven ahora entre README.md y los demás archivos?

Al ejecutar `git status`, la diferencia principal radica en cómo Git clasifica los archivos según hayan sido rastreados previamente o no.

---

c) Ahora agreguen TODOS los archivos restantes de una sola vez:

```
git add .
```

d) Ejecuten git status nuevamente. ¿Qué letra aparece ahora al lado de cada archivo? ¿Qué significa?

Al ejecutar `git status` en su versión corta (usando el comando `git status -s`), verás unas letras a la izquierda de los nombres de tus archivos. Estas letras son códigos de estado que te indican rápidamente qué está pasando con cada uno.

Ejercicio 4.2 – Eliminar un archivo del Staging Area

A veces agregamos un archivo por error. Practiquen cómo sacarlo del staging area:

a) Saquen el archivo notas.txt del staging area:

```
git rm --cached notas.txt
```

b) Ejecuten git status. ¿En qué estado quedó notas.txt? ¿Y los demás archivos?

Al ejecutar `git status` después de haber usado el comando `git add` específicamente sobre un archivo, verás una distinción clara en la terminal

---

c) Vuelvan a agregar notas.txt para que todos los archivos estén listos para el commit:

```
git add notas.txt
```

> **Resumen de estados**
> U (Untracked) = Git no rastrea ese archivo aún | A (Added) = El archivo está en el staging area, listo para commit | M (Modified) = El archivo fue modificado después del último commit

Parte 5: Realizando Commits

Ejercicio 5.1 – Su primer commit

Con todos los archivos en el staging area, van a guardar su primer instantánea del proyecto:

a) Ejecuten el siguiente comando para hacer su primer commit:

```
git commit -m 'feat: crear estructura inicial del proyecto'
```

b) ¿Qué mensaje les mostró la terminal después del commit? Escríbanlo:

[main (root-commit) 7a2b3c4] Tu mensaje de commit 1 file changed, 3 insertions(+) create mode 100644 notas.txt

---

Ejercicio 5.2 – Modificar, agregar y hacer un segundo commit

Ahora van a experimentar el flujo completo: modificar → agregar al staging → hacer commit.

a) Modifiquen el archivo index.html: agreguen un párrafo debajo del h1. Guarden el archivo.

b) Ejecuten git status. ¿Qué letra aparece al lado de index.html ahora? ¿Por qué?

M
La **M** significa **Modified** (Modificado).

---

c) Agreguen el archivo y hagan un commit con un mensaje usando conventional commits:

```
git add index.html
git commit -m 'su mensaje aquí usando conventional commits'
```

d) ¿Qué tipo de conventional commit usaron y por qué? (feat, fix, refactor, docs, style, chore)

Ejercicio 5.3 – Conventional Commits

Para cada situación, escriban qué tipo de conventional commit usarían:

Situación	Tipo de commit

Corregí un error en una función de Python	fix
Agregué una nueva página al sitio web	feat
Mejoré los nombres de las variables del código	Chore
Actualicé el archivo README.md con instrucciones	Docs
Cambié los colores del CSS de la página	style
Reorganicé las carpetas del proyecto	refactor

Parte 6: Explorando el Historial de Commits

Ejercicio 6.1 – Ver el historial

a) Ejecuten el siguiente comando para ver todos sus commits:

```
git log
```

b) Escriban la información que muestra para cada commit (ID, autor, fecha, mensaje):

commit 85a5e5fde93e5a0d06291d9b40f29443060e8a6 (HEAD -> principal)

Author: arenasvaleria030 <arenasvaleria030@gmail.com>

Date: Thu Apr 23 20:24:47 2026 -0500

su mensaje aquí usando conventional commits

commit f433424f042588513a3bd6417686f9a52e9074be

Author:arenasvaleria030 <arenasvaleria030@gmail.com>

Date: Thu Apr 23 20:24:47 2026 -0500

feat: crear estructura inicial del proyecto

---

c) Ahora prueben la versión resumida y compárenla:

```
git log --oneline
```

d) ¿Qué diferencia hay entre git log y git log --oneline? ¿Cuál les parece más útil y por qué?

El oneline sirve cuando solo se quiere ver el nombre de los commits y no los demas datos relacionados a este

---

Ejercicio 6.2 – Navegar entre commits

Van a viajar en el tiempo para ver cómo estaba su proyecto en el primer commit:

a) Copien el ID (los primeros caracteres) de su PRIMER commit del git log --oneline y ejecuten:

```
git checkout <peguen-aqui-el-ID-del-primer-commit>
```

b) ¿Qué pasó con los archivos del proyecto? ¿Se ven los cambios que hicieron después?

No aparecen como estaban antes de los cambios

---

c) Vuelvan a la rama principal ejecutando:

```
git checkout main
```

d) ¿Se recuperaron todos los archivos y cambios? ¿Qué aprendieron de esta experiencia?

Se puede revisar contenido anterior a los cambios sin miedo a perder lo que ya se habia hecho despues de estos

---

> **Importante**
> HEAD siempre apunta al último commit de la rama actual. Los cambios en commits anteriores se perderán si no se asocian a una nueva rama con: git checkout -b nombreRama

Parte 7: Reflexión Grupal

Ejercicio 7.1 – ¿Por qué usar Git?

Ahora que ya practicaron, respondan como grupo:

a) De los beneficios vistos en clase, ¿cuál les parece el más importante y por qué?

Permitir realizar cambios desde distintas fuentes sin temor a afectar negativamente el código.

---

b) Imaginen que están trabajando en un proyecto grupal de programación SIN Git. ¿Qué problemas podrían tener?

Poder integrar los cambios realizados por todo el equipo y contar con la posibilidad de retroceder a versiones anteriores en caso de que surja algún problema

---

c) Escriban en orden los comandos que aprendieron hoy, creando una "cheat sheet" de Git básico:

Comando	¿Qué hace?
`git init`	Para inicializar el repositorio
`git status`	Para revisar que archivos estan cambiados y si estan ya en el repositorio estos cambios o no
`git add .`	Para añadir archivos al stagin area
`git rm --cached`	Para quitar un archivo del stagin area
`git commit -m`	Para hacer commit de los archivos del stagin area
`git log`	Para revisar el historial de commit hechos anteriormente
`git log --oneline`	Para revisar el historial de commit de manera resumida
`git checkout`	Para cambiar entre commits

Criterios de Evaluación

- Instalación exitosa de Git y verificación con git --version
- Configuración correcta de user.name y user.email
- Creación del repositorio local con git init y archivos del proyecto
- Manejo correcto del staging area (git add, git rm --cached, git status)
- Realización de commits con mensajes usando conventional commits
- Exploración del historial con git log y navegación con git checkout
- Calidad de las reflexiones grupales y la cheat sheet final
- Participación activa de todos los integrantes del grupo